Need for Modifying Array Elements

- Data Processing
- User Input and Interaction
- Dynamic Data Structures
- Simulation and Modeling
- Error Correction and Handling
- Dynamic Input and Output

Array Traversal

Definition: Systematically visiting each element in an array.

Example (C Program – Traversal)

```C/C++
#include <stdio.h>

int main() {
    // Example array
    int myArray[] = {1, 2, 3, 4, 5};

    // Traverse the array and print each element
    for (int i = 0; i < 5; ++i) {
        printf("%d ", myArray[i]);
    }

    return 0;
}
```

Example (C Program – Modify Array Elements)

```C/C++
#include <stdio.h>
#define FACTOR 3

int main() {
    // Example array
    int myArray[] = {1, 2, 3, 4, 5};

    // Traverse the array and modify each element
    for (int i = 0; i < 5; ++i) {
        myArray[i] *= FACTOR;
        printf("%d ", myArray[i]);
    }

    return 0;
}
```

Array Concatenation

Definition: Array concatenation is the process of combining two arrays into a single array.

Example of Array Concatenation (C Program)

```C/C++
#include <stdio.h>

int main() {
    // Example arrays
    int myArray1[5] = {1, 2, 3, 4, 5};
    int myArray2[5] = {6, 7, 8, 9, 10};
    int combined[10];
```

```
    int i, j, l1 = 5, l2 = 5;

    j = 0;

    // Copy elements of first array
    for (i = 0; i < l1; i++) {
        combined[j] = myArray1[i];
        j++;
    }

    // Copy elements of second array
    for (i = 0; i < l2; i++) {
        combined[j] = myArray2[i];
        j++;
    }

    // Print combined array
    for (i = 0; i < 10; i++) {
        printf("%d ", combined[i]);
    }

    return 0;
}
```

Output

1 2 3 4 5 6 7 8 9 10

Array Copying

Uses of Array Copying

- Data Manipulation
- Function Parameters

- Backup and Undo Operations
- Algorithm Implementation
- Parallel Processing

Example (C Program – Array Copying)

```C/C++
#include <stdio.h>
#include <stdlib.h>

int main() {
    // Original array
    int originalArray[] = {1, 2, 3, 4, 5};
    int size = sizeof(originalArray) / sizeof(originalArray[0]);

    // Create a new array for copying
    int copiedArray[size];

    // Copy elements from original array to new array
    for (int i = 0; i < size; ++i) {
        copiedArray[i] = originalArray[i];
    }

    // Print copied array
    for (int i = 0; i < size; ++i) {
        printf("%d ", copiedArray[i]);
    }

    return 0;
}
```

Example (Extended Array Copy)

```C/C++
#include <stdio.h>
#include <stdlib.h>

int main() {
    // Original array
    int originalArray[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    int size = sizeof(originalArray) / sizeof(originalArray[0]);

    // Create a new array
    int copiedArray[size];

    // Copy elements
    for (int i = 0; i < size; ++i) {
        copiedArray[i] = originalArray[i];
    }

    // Display copied array
    for (int i = 0; i < size; ++i) {
        printf("%d ", copiedArray[i]);
    }

    return 0;
}
```

Output

1 2 3 4 5

(or for extended example: 1 2 3 4 5 6 7 8 9 10)

Array Insertion

Types of Insertion

- Inserting at a Specific Index
- Inserting at the End

Example (C Program – Insert at Specific Index)

```C/C++
#include <stdio.h>

int main() {
    // Original array
    int originalArray[] = {1, 2, 3, 4, 5};
    int size = sizeof(originalArray) / sizeof(originalArray[0]);

    // Element to insert
    int elementToInsert = 10;

    // Index at which to insert the element
    int indexToInsert = 2;

    // Check if index is valid
    if (indexToInsert < 0 || indexToInsert > size) {
        printf("Invalid index for insertion.\n");
        return 1;
    }

    // Create a new array with increased size
    int newSize = size + 1;
    int newArray[newSize];

    // Copy elements before the index
    for (int i = 0; i < indexToInsert; ++i) {
```

```
        newArray[i] = originalArray[i];
    }

    // Insert the new element
    newArray[indexToInsert] = elementToInsert;

    // Copy elements after the index
    for (int i = indexToInsert; i < size; ++i) {
        newArray[i + 1] = originalArray[i];
    }

    // Print original array
    printf("Original Array: ");
    for (int i = 0; i < size; ++i) {
        printf("%d ", originalArray[i]);
    }

    // Print array after insertion
    printf("\nArray after Insertion: ");
    for (int i = 0; i < newSize; ++i) {
        printf("%d ", newArray[i]);
    }

    return 0;
}
```

Output

Original Array: 1 2 3 4 5

Array after Insertion: 1 2 10 3 4 5

Key Concept (From Diagram)

- Original list: 2 4 1 7
- Insert element: 10
- New list: 2 4 1 10 7

Array Insertion (At the End)

Definition

Inserting at the end means adding a new element to the last position of an array.

Example (C Program – Insert at End)

```C/C++
#include <stdio.h>

int main() {
    // Original array
    int originalArray[] = {1, 2, 3, 4, 5};
    int size = sizeof(originalArray) / sizeof(originalArray[0]);

    // Element to insert
    int elementToInsert = 10;

    // Create a new array with increased size
    int newSize = size + 1;
    int newArray[newSize];

    // Copy elements from original array
    for (int i = 0; i < size; ++i) {
        newArray[i] = originalArray[i];
    }

    // Insert the new element at the end
```

```c
    newArray[size] = elementToInsert;

    // Print original array
    printf("Original Array: ");
    for (int i = 0; i < size; ++i) {
        printf("%d ", originalArray[i]);
    }

    // Print new array
    printf("\nArray after inserting at end: ");
    for (int i = 0; i < newSize; ++i) {
        printf("%d ", newArray[i]);
    }

    return 0;
}
```

Output

Original Array: 1 2 3 4 5

Array after inserting at end: 1 2 3 4 5 10

Key Concept

- No shifting of elements is required
- Simply copy existing elements and add the new element at the last index

Array Deletion

Types of Deletion

- Deleting by Index
- Deleting by Value

Concept (from diagram)

- Original list: `2 4 1 10 7`
- Delete element: `10`
- New list: `2 4 1 7`

Delete Element at Specific Index (C Program)

```C/C++
#include <stdio.h>

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int size = 5;
    int index = 2;  // delete element at index 2

    if (index < 0 || index >= size) {
        printf("Invalid index\n");
        return 1;
    }

    // Shift elements to left
    for (int i = index; i < size - 1; i++) {
        arr[i] = arr[i + 1];
    }
```

```
    size--; // reduce size

    // Print updated array
    printf("Array after deletion: ");
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Output

1 2 4 5

Delete Specific Element (by Value) (C Program)

```C/C++
#include <stdio.h>

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int size = 5;
    int value = 3;  // element to delete
    int found = -1;

    // Find the element
    for (int i = 0; i < size; i++) {
        if (arr[i] == value) {
            found = i;
            break;
```

```
        }
    }

    if (found == -1) {
        printf("Element not found\n");
        return 1;
    }

    // Shift elements
    for (int i = found; i < size - 1; i++) {
        arr[i] = arr[i + 1];
    }

    size--;

    // Print updated array
    printf("Array after deletion: ");
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Output

1 2 4 5

Delete Specific Element (General Function Version)

```C/C++
#include <stdio.h>

void deleteByValue(int arr[], int *size, int value) {
    int i, j;

    for (i = 0; i < *size; i++) {
        if (arr[i] == value) {
            for (j = i; j < *size - 1; j++) {
                arr[j] = arr[j + 1];
            }
            (*size)--;
            return;
        }
    }

    printf("Element not found\n");
}

int main() {
    int arr[] = {2, 4, 1, 10, 7};
    int size = 5;

    deleteByValue(arr, &size, 10);

    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

More Operations

- Finding the Maximum and Minimum
- Reversing an Array
- Searching for an Element

- Calculating the Sum and Average

---

1. Finding Maximum and Minimum in an Array

```C/C++
#include <stdio.h>

int main() {
    int arr[] = {12, 45, 7, 89, 23};
    int size = sizeof(arr) / sizeof(arr[0]);

    int max = arr[0];
    int min = arr[0];

    for (int i = 1; i < size; i++) {
        if (arr[i] > max) {
            max = arr[i];
        }
        if (arr[i] < min) {
            min = arr[i];
        }
    }

    printf("Maximum element = %d\n", max);
    printf("Minimum element = %d\n", min);

    return 0;
}
```

---

2. Reversing an Array

```c
#include <stdio.h>

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int size = sizeof(arr) / sizeof(arr[0]);

    int temp;
    for (int i = 0; i < size / 2; i++) {
        temp = arr[i];
        arr[i] = arr[size - 1 - i];
        arr[size - 1 - i] = temp;
    }

    printf("Reversed array: ");
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

---

3. Searching for an Element in an Array

```c
#include <stdio.h>

int main() {
    int arr[] = {10, 20, 30, 40, 50};
    int size = sizeof(arr) / sizeof(arr[0]);
    int key = 30;
    int found = 0;
```

```
    for (int i = 0; i < size; i++) {
        if (arr[i] == key) {
            printf("Element %d found at index %d\n", key, i);
            found = 1;
            break;
        }
    }

    if (!found) {
        printf("Element not found\n");
    }

    return 0;
}
```

---

4. Calculating Sum and Average of Array Elements

```C/C++
#include <stdio.h>

int main() {
    int arr[] = {10, 20, 30, 40, 50};
    int size = sizeof(arr) / sizeof(arr[0]);

    int sum = 0;
    float average;

    for (int i = 0; i < size; i++) {
        sum += arr[i];
    }

    average = (float)sum / size;
```

```
    printf("Sum = %d\n", sum);
    printf("Average = %.2f\n", average);

    return 0;
}
```

---

5. Combined Program for All Operations

```C/C++
#include <stdio.h>

int main() {
    int arr[] = {12, 45, 7, 89, 23};
    int size = sizeof(arr) / sizeof(arr[0]);

    int max = arr[0], min = arr[0], sum = 0;
    float average;
    int key = 45, found = 0;

    for (int i = 0; i < size; i++) {
        if (arr[i] > max)
            max = arr[i];
        if (arr[i] < min)
            min = arr[i];
        sum += arr[i];
    }

    average = (float)sum / size;

    printf("Maximum = %d\n", max);
    printf("Minimum = %d\n", min);
```

```c
    printf("Sum = %d\n", sum);
    printf("Average = %.2f\n", average);

    for (int i = 0; i < size; i++) {
        if (arr[i] == key) {
            printf("Element %d found at index %d\n", key, i);
            found = 1;
            break;
        }
    }

    if (!found) {
        printf("Element not found\n");
    }

    int temp;
    for (int i = 0; i < size / 2; i++) {
        temp = arr[i];
        arr[i] = arr[size - 1 - i];
        arr[size - 1 - i] = temp;
    }

    printf("Reversed array: ");
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Time Complexity

Operation	Time Complexity	Notes
Accessing an element by index (Random Access)	O(1)	Constant time operation
Inserting or deleting at the end of the array	O(1) on average	Constant time on average, assuming space is available
Inserting or deleting at the beginning or middle	O(n)	May require shifting elements, resulting in linear time
Searching in an unsorted array	O(n)	Linear time in the worst case
Searching in a sorted array (Binary Search)	O(log n)	Logarithmic time with efficient search algorithm
Sorting an array (Quicksort, Mergesort, etc.)	O(n log n)	Average time complexity for efficient sorting algorithms
Copying an array	O(n)	Linear time, requires visiting each

Space Complexity

Operation	Space Complexity	Notes
Accessing an element by index (Random Access)	O(1)	Constant space, as no additional memory is required
Inserting or deleting at the end of the array	O(1)	Constant space, assuming space is available
Inserting or deleting at the beginning or middle	O(1)	Constant space, as the number of elements remains the same
Searching in an unsorted array	O(1)	Constant space, as no additional memory is required
Searching in a sorted array (Binary Search)	O(1)	Constant space, as no additional memory is required
Sorting an array (Quicksort, Mergesort, etc.)	O(log n)	Logarithmic space complexity for recursive algorithms
Copying an array	O(n)	Linear space, requires additional memory for a copy of the array

Array Operations (Advanced Concepts)

1. Indexing and Slicing

- Access individual elements or subarrays by specifying their position or range within the array.

---

2. Element-wise Operations

- Perform operations on corresponding elements of two arrays.
- Examples: addition, subtraction, multiplication, etc.
- No need for explicit loops.

---

3. Aggregation and Reduction

- Combine array values using functions like:
  - `sum`
  - `mean`
  - `max`
  - `min`
- Reduction operations reduce the size (dimension) of the array.

---

4. Broadcasting

- Automatically expands smaller arrays to match larger arrays.
- Helps perform operations on arrays of different sizes without manual resizing.